IoT MINI-PROJECT REPORT

# Smart Water Distribution and Leakage Detection

*An IoT, MQTT, Node-RED and machine-learning system that detects leaks, sensor faults and cyber-attacks in a water network and acts on them automatically*


**Team 2**

[Student name 1] ([Roll number])

[Student name 2] ([Roll number])

[Student name 3] ([Roll number])

B.Tech Computer Science and Engineering, Semester 7

Guide: [Guide name]

[College name], [Department]


November 2026

# Abstract

Water-distribution networks lose water through leaks, and their control systems are exposed to faulty sensors and, increasingly, to cyber-attacks. This project builds a complete IoT monitoring and automation system for such a network, following the prescribed architecture: IoT node → MQTT broker → Node-RED → database → dashboard → machine-learning analytics → decision engine → MQTT broker → actuators.

The system runs on **two laptops** that communicate only through the online **HiveMQ** MQTT broker. One laptop is the **IoT device**: it replays real recordings from the **WDSEventDB** water-distribution testbed dataset (4 pressure, 4 flow, 5 pump-drive and 2 valve signals) and operates the actuators. The other is the **cloud**: Node-RED validates and stores every reading in SQLite and serves a live dashboard, and a Python decision engine classifies each reading as normal, leak, sensor fault or cyber-attack.

Nine machine-learning models were compared, and the best of each family was kept: **k-NN** (traditional), **Extra Trees** (ensemble) and a **CNN-LSTM** (deep learning). On a held-out test set the deployed Extra Trees model reaches **99.87 % accuracy and a macro F1 of 0.996** with a 0.06 % false-alarm rate. Removing all pump and valve signals lowers macro F1 only to 0.989, so the models detect events from the physical pressure and flow measurements.

The decision engine confirms events by voting over recent predictions, grades leak severity, locates the suspected zone and sends commands that reduce pump speed, isolate the leaking section, flag a faulty sensor or put the controller into safe mode. Advanced features include **HMAC-signed control commands** with replay protection (the device rejects forged commands on the public broker), **explainable alerts**, automatic clock-offset correction between the two laptops, live model switching, scenario injection and a downloadable incident report. In a test over the real HiveMQ broker the system detected all 12 events with none missed, with about 1.6 s from sensor reading to actuator acknowledgement.

# Contents

Abstract ..... 1
1. Introduction ..... 3
1.1 Problem statement ..... 3
1.2 Objectives ..... 3
1.3 Scope ..... 3
2. System Architecture ..... 4
2.1 Two-laptop deployment ..... 4
3. Dataset ..... 6
4. Implementation ..... 7
4.1 IoT device ..... 7
4.2 MQTT communication ..... 7
4.3 Node-RED flow processing ..... 7
4.4 Database ..... 8
4.5 Dashboard ..... 8
5. Analytics and Machine Learning ..... 10
5.1 Feature engineering ..... 10
5.2 Training and test split ..... 10
5.3 Model selection ..... 10
5.4 Deployed models ..... 11
5.5 Explainability and a check for shortcut learning ..... 12
6. Decision and Automation ..... 13
6.1 Hybrid detection ..... 13
6.2 Stability (avoiding unnecessary switching) ..... 13
6.3 Leak severity and location ..... 13
6.4 Control actions ..... 14
6.5 Closing the loop on recorded data ..... 14
7. Advanced Features ..... 15
7.1 Authenticated control commands ..... 15
7.2 Explainable alerts ..... 15
7.3 Further features ..... 15
8. Results and Performance Evaluation ..... 17
9. Limitations ..... 19
10. Conclusion and Future Work ..... 20
References ..... 21
Appendix A: How to Run ..... 22

# 1. Introduction

## 1.1 Problem statement

Water-distribution networks require continuous monitoring to identify leakage, abnormal consumption and inefficient pump operation. Flow, pressure, tank-level and pump-status data can be used to study the behaviour of the network. The task is to analyse normal and abnormal water-use patterns, identify leakage and, where the data allows, classify the type or severity of the problem, then generate an alert and take a suitable control action: operating pumps or valves, or isolating a section where a leak is suspected.

## 1.2 Objectives

- Stream sensor data from an IoT node over MQTT and process it in Node-RED.
- Store every reading in a database and visualise the network live on a Node-RED dashboard.
- Detect and classify four conditions with machine learning: **normal, leak, sensor fault, cyber-attack**.
- Grade leak severity, locate the suspected zone and identify the faulty sensor.
- Act automatically in a closed loop (pumps, isolation valves, safe mode) and restore normal operation afterwards.
- Evaluate ML, computational and closed-loop performance, as required by the project architecture.
- Deploy the device and the cloud on two separate laptops connected through an online broker.

## 1.3 Scope

The IoT device replays recorded data from a real laboratory testbed instead of reading physical sensors. The effect of control actions on the network is therefore approximated (Section 6.5). The dataset contains no tank-level sensor, so pump status is taken from the pump drive (VFD) signals.

# 2. System Architecture

Figure 1 shows the implemented architecture. The numbered arrows correspond to the steps of the prescribed project architecture.

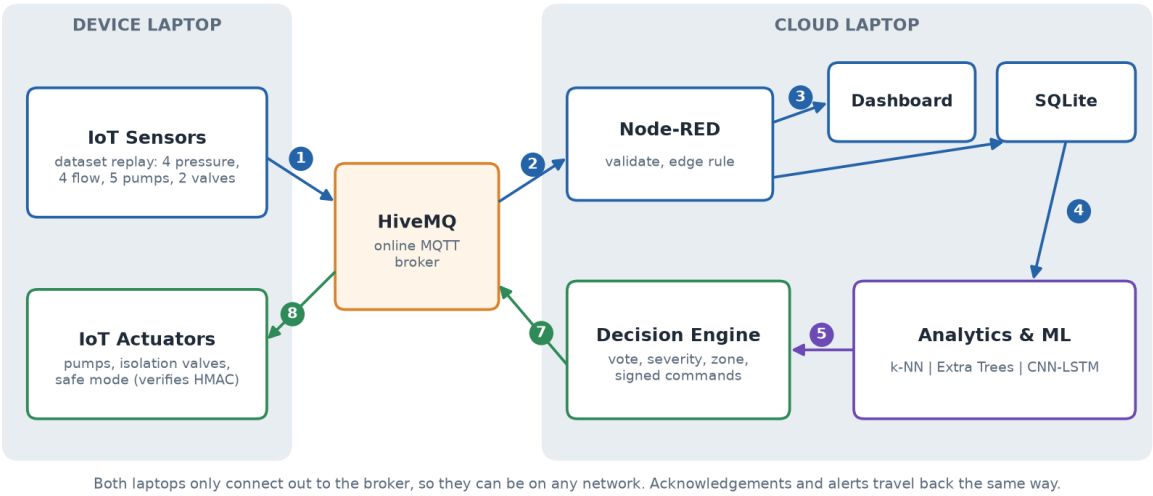



*Figure 1: System architecture on two laptops connected through the HiveMQ broker*

**Table 1: Architecture steps and their implementation**

| Step | What happens | Implementation |
|---|---|---|
| 1 | The IoT device publishes each sensor reading | `iot_device.py` on the device laptop, topic `…/sensors/telemetry` |
| 2 | The broker delivers it to Node-RED | HiveMQ public broker; Node-RED *mqtt in* node |
| 3 | Node-RED validates, stores and displays it | function nodes, SQLite, Node-RED Dashboard |
| 4 | Analytics reads new readings from the database | `engine/decision_engine.py` polls SQLite |
| 5 | ML classifies and the decision engine decides | k-NN / Extra Trees / CNN-LSTM, voting, diagnosis |
| 7 | A signed control command is published | topic `…/control/cmd` |
| 8 | The actuators execute and acknowledge it | `iot_device.py`, topic `…/actuators/state` |

## 2.1 Two-laptop deployment

The two laptops never connect to each other directly. Each only opens an outgoing connection to `broker.hivemq.com`, so they can be on different networks without port forwarding. The device laptop needs only Python: the MQTT client library and the recordings are bundled in its package. The cloud laptop runs Node-RED, SQLite, the dashboard, the ML models and the decision engine.

Because the public broker is shared with other users, every topic starts with a random project identifier (`wds-t2-906e031a`) and every client uses a unique client ID. The identifier only separates our traffic; it is not security. Section 7.1 describes how commands are authenticated.

**Table 2: Technologies used**

| Technology | Used for | Reason |
|---|---|---|
| Python 3 | IoT device, ML training, decision engine | Strong ecosystem for data and ML |
| pandas, NumPy | Dataset handling, feature engineering | Standard tabular-data tools |
| scikit-learn | k-NN, Extra Trees (and the other candidates) | Reliable, well-documented ML library |
| PyTorch | CNN-LSTM deep-learning model | Flexible deep-learning framework |
| MQTT (paho-mqtt) | All device ↔ cloud communication | Lightweight publish/subscribe protocol designed for IoT |
| HiveMQ public broker | Online MQTT broker | Free, no account, reachable from any network |
| Node-RED 5 | Flow processing and integration | Visual flow-based integration, required by the brief |
| Node-RED Dashboard | Web dashboard | Charts, gauges and controls, required by the brief |
| SQLite (node:sqlite) | Database | File-based, zero setup; built into Node.js 24 |
| HMAC-SHA256 | Command authentication | Standard message authentication (RFC 2104) |

# 3. Dataset

The project uses **WDSEventDB**, a database of events recorded on a laboratory water-distribution testbed. Each row is one reading with 15 signals and a binary label (0 = normal, 1 = event).

**Table 3: Signals in each reading**

| Group | Signals | Meaning |
|---|---|---|
| Pressure | Pressure 1 Out, 2 Out, 3 In, 4 In | Bar. The inlet sensors can read slightly negative in normal operation (suction side). |
| Flow | Water Flow 1-4 | Flow at four points of the network |
| Pumps | VFD 1, 2, 3, 4-1, 4-2 | Pump speed via variable-frequency drives |
| Valves | Analog Valve 1, 2 | Valve opening in % |

**Table 4: Recordings**

| Folder | Files | Readings | Event |
|---|---|---|---|
| Clean | CleanData | 9,743 | None |
| Leakage | LeakEvent1-3 | ~1,370 each | Normal operation, then a leak |
| Sensor Failure | SensorEvent1, 23, 45, 67 | ~1,360 each | 1: P1; 23: P2 then a small P4 fault; 45: P3 twice; 67: P4 twice |
| Cyberattack | CyberEvent1-4 | 5,869 | Four attacks: valve 2 forced to ~100 %, pump 1 slowed |

The binary label is combined with the folder name to create a **four-class target**: 0 Normal, 1 Leak, 2 Sensor fault, 3 Cyber-attack. In total there are 25,165 readings.

The network is divided into four zones, each watched by one pressure and one flow sensor and protected by one isolation valve: zone A (P1/F1, main supply), B (P2/F2), C (P3/F3) and D (P4/F4).

# 4. Implementation

## 4.1 IoT device

`iot_device.py` runs on the device laptop and depends only on the Python standard library plus the bundled paho-mqtt client. It has three roles:

- **Sensors:** it plays a playlist of recordings (clean → leak → clean → sensor fault → leak → cyber-attack → leak → sensor fault) at a configurable rate (default 5 readings/s) and publishes each reading as JSON with QoS 0.
- **Actuators:** it keeps the state of the pumps, the four isolation valves and the controller mode, executes commands after a simulated 300 ms actuator delay, and acknowledges each one.
- **Results:** it prints the cloud's alerts and the confirmed system state, and saves them to `results_log.txt`.

A Last Will message (`online: false`) is registered with the broker, so the dashboard shows the device as OFFLINE if it disconnects unexpectedly.

## 4.2 MQTT communication

**Table 5: MQTT topics (all prefixed with wds-t2-906e031a/)**

| Topic | Direction | Content | QoS |
|---|---|---|---|
| sensors/telemetry | device → cloud | One reading: 15 signals, timestamp, sequence number | 0 |
| control/cmd | cloud → device | Signed control command | 1 |
| actuators/state | device → cloud | Acknowledgement and actuator state | 1 |
| actuators/state/current | device → cloud | Current state (retained) | 1 |
| analytics/prediction | cloud → all | ML result for every reading | 1 |
| alerts | both | Alerts from engine, Node-RED edge rule and device | 1 |
| sim/control, sim/status | both | Scenario injection; online status (retained, Last Will) | 1 |
| engine/config, engine/kpi | cloud | Live settings; closed-loop KPIs | 1 |
| engine/ping, device/pong | both | Clock-offset and round-trip measurement | 0 |

Telemetry uses QoS 0 because readings are frequent and losing one is harmless; commands, acknowledgements and alerts use QoS 1 so that they are delivered at least once. Retained messages let a newly opened dashboard show the current actuator and device state immediately.

## 4.3 Node-RED flow processing

The Node-RED project has five flow tabs (141 nodes). The ingest flow (Figure 2) validates each reading against physical limits, adds the receive time and a quality flag, raises an immediate **edge alert** for impossible values, inserts the reading into SQLite and forwards it to the dashboard. Other flows store

predictions, alerts, commands and acknowledgements, build the control panel, compute database statistics and serve the incident report.

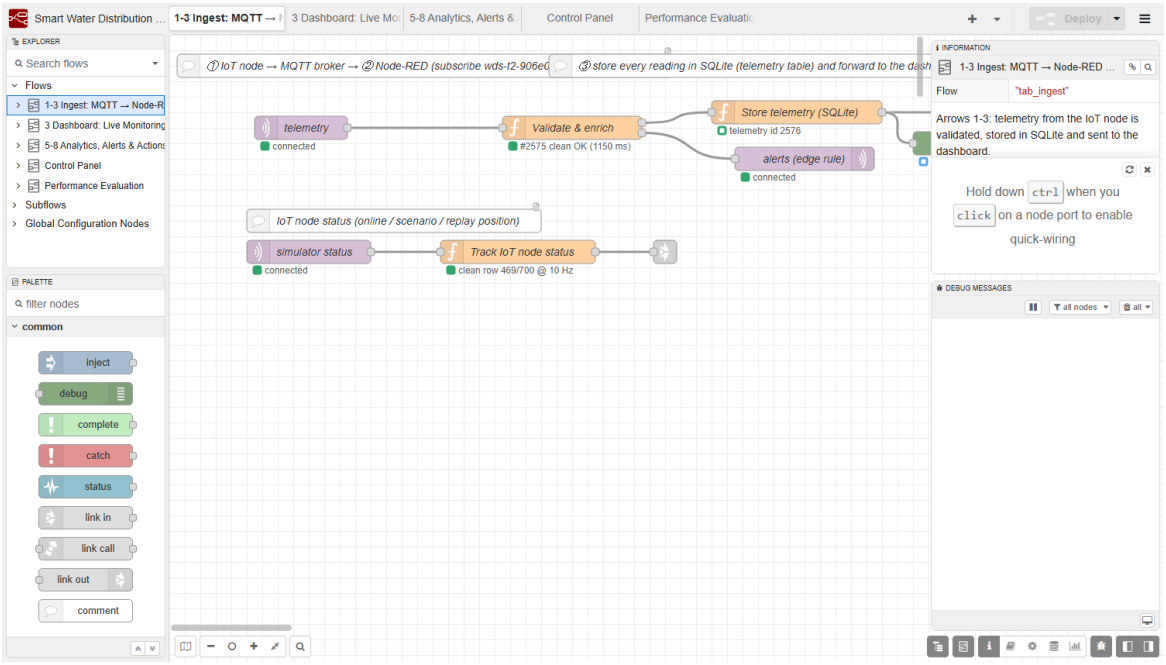



*Figure 2: Node-RED ingest flow: MQTT input, validation, SQLite storage and edge-rule alerts*

Node-RED accesses SQLite through Node.js's built-in `node:sqlite` module, opened once in `settings.js` and shared with function nodes. This avoids the native `sqlite3` add-on, which needs a C++ compiler on Windows.

## 4.4 Database

**Table 6: SQLite tables**

| Table | Written by | Content |
|---|---|---|
| telemetry | Node-RED | Every reading, receive time, quality flag, ground-truth label (for scoring only) |
| predictions | Node-RED (from engine) | Class, confidence, confirmed state, severity, zone, latency |
| alerts | Node-RED | Alerts from the engine, the edge rule and the device |
| actions | Node-RED | Every command with acknowledgement time, response time and effect |
| kpi_log | Node-RED | Closed-loop KPI snapshots every 10 s |

The database runs in WAL mode, so Node-RED can write while the decision engine reads. Node-RED performs all writes; the engine only reads the telemetry table.

## 4.5 Dashboard

The dashboard has four pages: **Live Monitoring** (pressure and flow charts, gauges, pump and valve positions), **Analytics & Alerts** (confirmed state, class probabilities, prediction timeline, alerts, actuator state, action log), **Control Panel** (automation settings, manual override, scenario injection) and

**Performance** (live KPIs, model comparison, feature importance, database statistics, incident report download).




*Figure 3: Live Monitoring page during a leak*

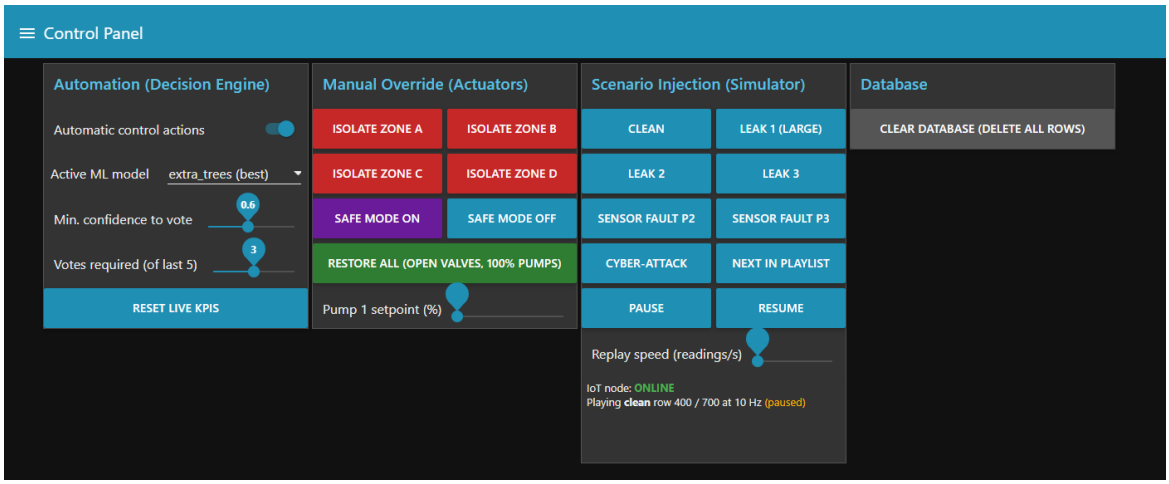



*Figure 4: Control Panel: automation settings, manual override and scenario injection*

# 5. Analytics and Machine Learning

## 5.1 Feature engineering

A single reading says little; a leak shows up as a change compared with the recent past. For every reading, 53 features are computed from the last 60 readings, looking only backwards in time so that the online engine computes exactly what the model saw in training:

- the 15 raw signals;
- for each pressure and flow: 10-reading rolling mean and standard deviation, deviation from the 60-reading mean, and first difference (32 features);
- hydraulic indicators: head P1−P2 and P4−P3, minimum pressure, total outflow, pump-1 speed per unit flow, valve difference.

The CNN-LSTM instead receives the raw sequence of the last 30 readings and learns its own temporal features.

## 5.2 Training and test split

Each recording is divided into contiguous blocks of 60 readings, and 30 % of the blocks of every class form the test set (17,622 training and 7,543 test readings). Neighbouring readings are almost identical, so a random row split would put near-copies of test readings in the training set and overstate accuracy; block splitting prevents this. Class imbalance (15,264 normal training readings against 408 sensor-fault readings) is handled with balanced class weights.

The label is only the training target. Model inputs are the 15 sensor signals and features computed from them; we verified that faking or removing the label column leaves every prediction unchanged.

## 5.3 Model selection

Nine candidates from the three families in the architecture were trained on the same split. The best model of each family by macro F1 was kept for deployment.

**Table 7: Candidate models (test set)**

| Model | Family | Accuracy | Macro F1 | False alarms | Kept |
|---|---|---|---|---|---|
| Logistic Regression | Traditional | 0.967 | 0.899 | 3.11 % | |
| k-NN (k = 7) | Traditional | 0.989 | 0.970 | 0.34 % | **Yes** |
| Decision Tree | Traditional | 0.964 | 0.899 | 2.83 % | |
| Gaussian Naive Bayes | Traditional | 0.347 | 0.499 | 75.1 % | |
| Random Forest | Ensemble | 0.997 | 0.987 | 0.08 % | |
| Extra Trees | Ensemble | 0.999 | 0.996 | 0.06 % | **Yes** |
| Gradient Boosting | Ensemble | 0.996 | 0.985 | 0.22 % | |
| MLP | Deep learning | 0.984 | 0.945 | 1.37 % | |
| CNN-LSTM | Deep learning | 0.987 | 0.950 | 1.38 % | **Yes** |

Naive Bayes fails because it assumes independent features, while ours are strongly correlated (a signal and its rolling mean, for example). The CNN-LSTM's lead over the MLP is within run-to-run training variation (it scored 0.944 when retrained), but it was kept because it learns from the time sequence directly and has the lowest inference latency.

## 5.4 Deployed models

**Table 8: Deployed models: ML and computational performance (test set)**

| Metric | k-NN | Extra Trees | CNN-LSTM |
|---|---|---|---|
| Accuracy | 0.9889 | **0.9987** | 0.9852 |
| Precision (macro) | 0.9754 | **0.9949** | 0.9130 |
| Recall (macro) | 0.9656 | **0.9975** | 0.9892 |
| F1 (macro) | 0.9700 | **0.9962** | 0.9440 |
| False-alarm rate | 0.34 % | **0.06 %** | 1.57 % |
| Recall: normal / leak | 0.997 / 0.923 | 0.999 / 0.991 | 0.984 / 0.987 |
| Recall: sensor fault / cyber-attack | 0.965 / 0.978 | 1.000 / 1.000 | 0.985 / 1.000 |
| Training time | 0.04 s | 0.25 s | 11.76 s |
| Model size | 3,789 KB | 5,679 KB | 109 KB |
| Inference latency per reading* | 6.9 ms | 13.8 ms | **0.8 ms** |

*Latency includes feature computation, exactly as the online engine uses the model; the CNN-LSTM needs no hand-made features and is therefore the fastest. Extra Trees has the best macro F1 and is the default model. The other two can be switched in live from the dashboard.

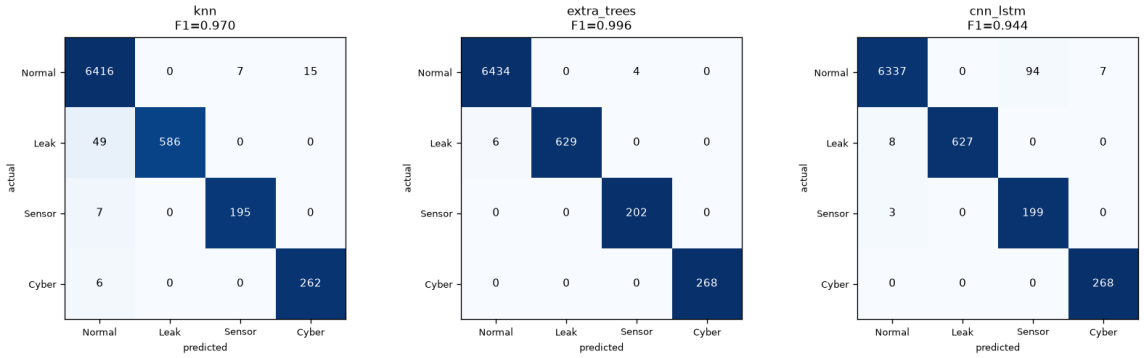



*Figure 5: Confusion matrices of the three deployed models (test set)*

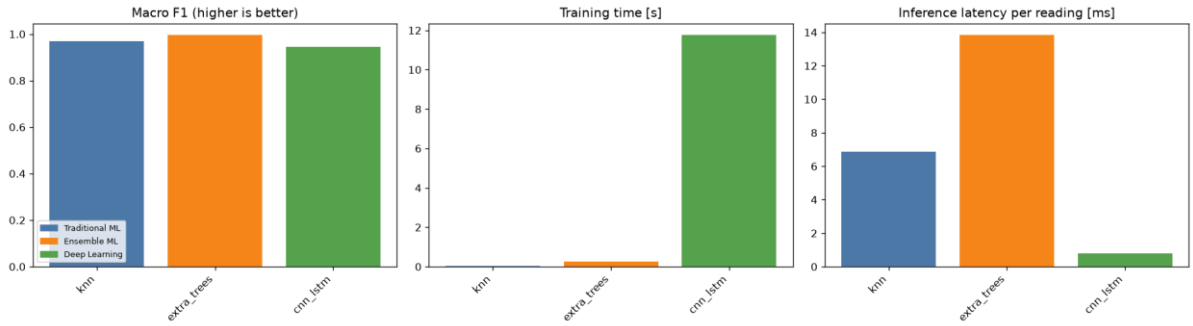



*Figure 6: Macro F1, training time and inference latency of the deployed models*

## 5.5 Explainability and a check for shortcut learning

Figure 7 shows the features Extra Trees relies on most. Valve 2 ranks first, which is expected because the cyber-attacks force valve 2 to about 100 %. Several pump-drive settings also rank high, and in the leak recordings both analog valves open further when the leak begins (for example, valve 1 rises from 49 % to 72 % in LeakEvent1). This raised a concern that the model might partly recognise each recording's valve and pump settings instead of the leak itself.

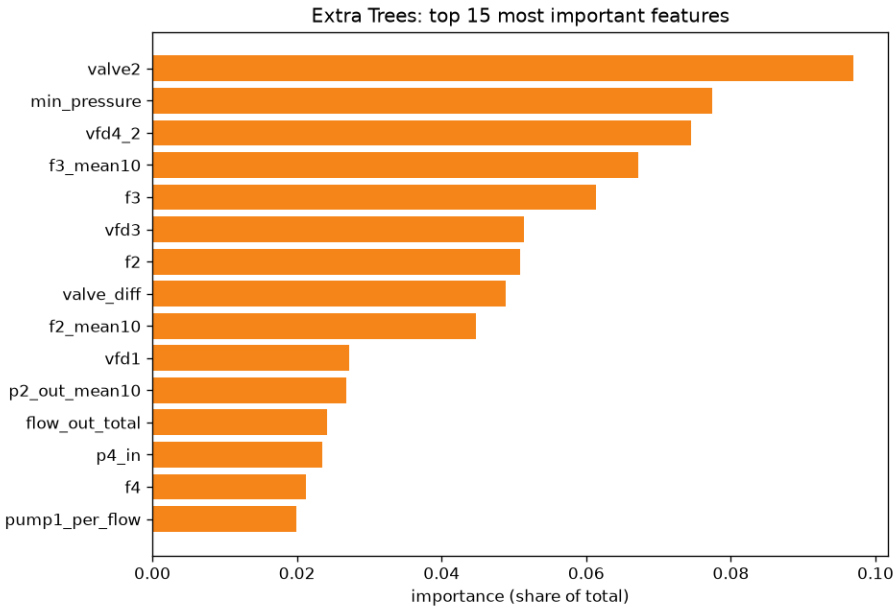



*Figure 7: Top 15 features of the Extra Trees model*

To test this, Extra Trees was retrained **without all nine pump and valve features**. Macro F1 fell only from 0.996 to **0.989**, leak recall stayed at 0.989, and sensor-fault and cyber-attack recall stayed at 1.000. The models therefore detect events mainly from the physical pressure and flow measurements. For the same reason, the evidence shown with leak and sensor-fault alerts uses only pressure and flow signals (Section 7.2).

# 6. Decision and Automation

The decision engine turns per-reading predictions into confirmed events and control actions (Figure 8).

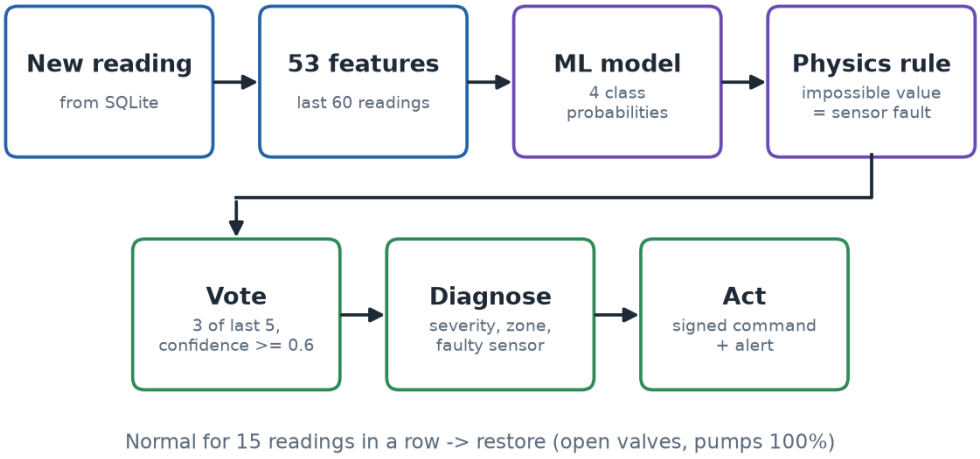



*Figure 8: Decision pipeline for every new reading*

## 6.1 Hybrid detection

The ML prediction is combined with a physical rule: a pressure outside −4 to 15 bar is always treated as a sensor fault. The lower limit is −4 bar rather than 0, because the inlet sensors legitimately read down to about −2.8 bar in normal operation.

## 6.2 Stability (avoiding unnecessary switching)

- An event is confirmed only when **3 of the last 5** predictions agree with confidence ≥ 0.6.
- It is cleared only after **15 consecutive normal** readings, followed by a restore command.
- Leak severity can escalate during an event but never de-escalates, and identical alerts are rate-limited.

This hysteresis stops valves from switching repeatedly on noisy predictions, at the cost of a detection delay of a few readings, which is measured in Section 8.

## 6.3 Leak severity and location

For each zone, a leak score is computed as the relative increase in flow plus the relative drop in pressure, compared with a baseline of recent normal readings. The leak index is the highest zone score, and that zone is the suspected location. Severity thresholds were learned from the data as the 33rd and 66th percentiles of the leak index over all leak readings: **minor < 0.257 ≤ moderate < 1.201 ≤ severe**. LeakEvent1 (index 1.26) is located in zone C; LeakEvent2 and 3 (0.24 and 0.29) in zone B.

## 6.4 Control actions

**Table 9: Event handling**

| Event | Alert | Control action |
|---|---|---|
| Leak, minor | Warning | Pump 1 to 85 % (lower pressure reduces leakage) |
| Leak, moderate | Critical | Pump 1 to 70 % and isolate the suspected zone |
| Leak, severe | Critical | Pump 1 to 50 % and isolate the suspected zone |
| Sensor fault | Warning + faulty sensor | Flag the sensor for maintenance; no hydraulic action |
| Cyber-attack | Critical | Controller to safe mode (remote setpoints locked) |
| Back to normal | Info | Restore: open valves, pumps to 100 %, clear flags |
| Pump inefficiency | Warning | Advisory only, when pump-1 speed per unit flow exceeds 1.6× normal |

With **auto mode off** the engine sends no commands and publishes recommended actions as alerts, so an operator decides (human in the loop). All thresholds are in `config.json`, and the main ones can be changed live from the dashboard.

## 6.5 Closing the loop on recorded data

A recording cannot react to a closed valve, so the device approximates the effect: when the leaking section is isolated, or safe mode is switched on during an attack, it skips the rest of that anomalous run after a 2 s valve travel time. The network then returns to normal, the engine observes normal readings and sends restore. Pump-speed changes and sensor flags are recorded but do not alter the replayed data.

# 7. Advanced Features

## 7.1 Authenticated control commands

The broker is public, so anyone who knew the topic names could publish a command such as "close valve". Every command therefore carries an **HMAC-SHA256** signature over its identifier, timestamp, action, target and value, computed with a key shared by the cloud and the device. The device recomputes the signature and also rejects commands it has already executed or that are more than 5 minutes old (replay protection). A rejected command produces a FORGED_COMMAND alert on the dashboard.

**Table 10: Security tests over the real broker**

| Attack | Result |
|---|---|
| Command without signature | Rejected, alert raised |
| Command signed with a guessed key | Rejected |
| Genuine command with its target changed | Rejected (signature mismatch) |
| Genuine command sent again (replay) | Rejected (already executed) |
| Signed command 10 minutes old | Rejected (too old) |
| Genuine engine and dashboard commands | Accepted |

`tools/attack_demo.py` plays the attacker for a live demonstration. The scheme protects commands only; telemetry is not signed, and the key is stored in the project files (Section 9).

## 7.2 Explainable alerts

Each leak, sensor-fault and cyber-attack alert states the evidence behind it: the three signals that changed most from normal operation, for example "p3_in 2.052 → 0.819 (−57 %), p2_out 1.729 → 0.823 (−50 %), p4_in 2.142 → 1.065 (−48 %)". Leak and sensor-fault evidence uses pressure and flow only; cyber-attack evidence also includes pump and valve setpoints, because manipulating them is the attack. The Performance page shows the global feature importance (Figure 7).

## 7.3 Further features

- **Four-class detection** with severity grading, zone localisation and faulty-sensor identification, not only leak yes/no.
- **Hybrid ML and physics rule**, with voting and hysteresis for stable control.
- **Clock-offset correction:** every 5 s the engine pings the device and estimates the difference between the laptops' clocks (NTP-style), so latencies measured across two clocks stay correct. With the device clock shifted by 7 s on purpose, the engine measured 6,985 ms.
- **Live model switching** between the three models, and auto/manual mode, from the dashboard.
- **Scenario injection:** dashboard buttons make the device play a leak, sensor fault or cyber-attack on demand.

- **Edge processing in Node-RED:** immediate alerts for physically impossible readings, before any ML.
- **Incident report:** a CSV of every alert and action, downloadable from the dashboard.
- **Device status** through MQTT Last Will and retained messages.

# 8. Results and Performance Evaluation

Performance is evaluated in the three categories of the project architecture: ML performance and computational performance (Table 8) and closed-loop performance (Table 11).

**Table 11: Closed-loop performance over the real HiveMQ broker**

| KPI | Value |
|---|---|
| True events / detected / missed / false | 12 / 12 / 0 / 1 (one brief false sensor-fault event) |
| Online accuracy | 99.72 % (optimistic; see note below) |
| Detection delay (true onset → confirmed) | ~5 readings on average (3-11), ~940 ms |
| Broker round trip (cloud → HiveMQ → device → cloud) | ~412 ms at best |
| Response time (command → acknowledgement at the cloud) | ~1,060 ms, including the 300 ms simulated actuator delay |
| End-to-end latency (sensor reading → acknowledgement) | ~1,560 ms |
| Leaks contained / attacks blocked | 3 / 1 |
| ML inference per reading (Extra Trees) | ~13 ms |

These figures were measured at 10 readings/s with the device and cloud programs on one laptop, connected through the real HiveMQ broker, with the device clock deliberately shifted by 7 s. A two-laptop run depends on both internet connections and will give different latencies.

The online accuracy is optimistic, because the demo replays the same recordings the models were trained on (70 % of each recording is in the training set). The held-out test-set results in Section 5 are the fair measure of model accuracy.

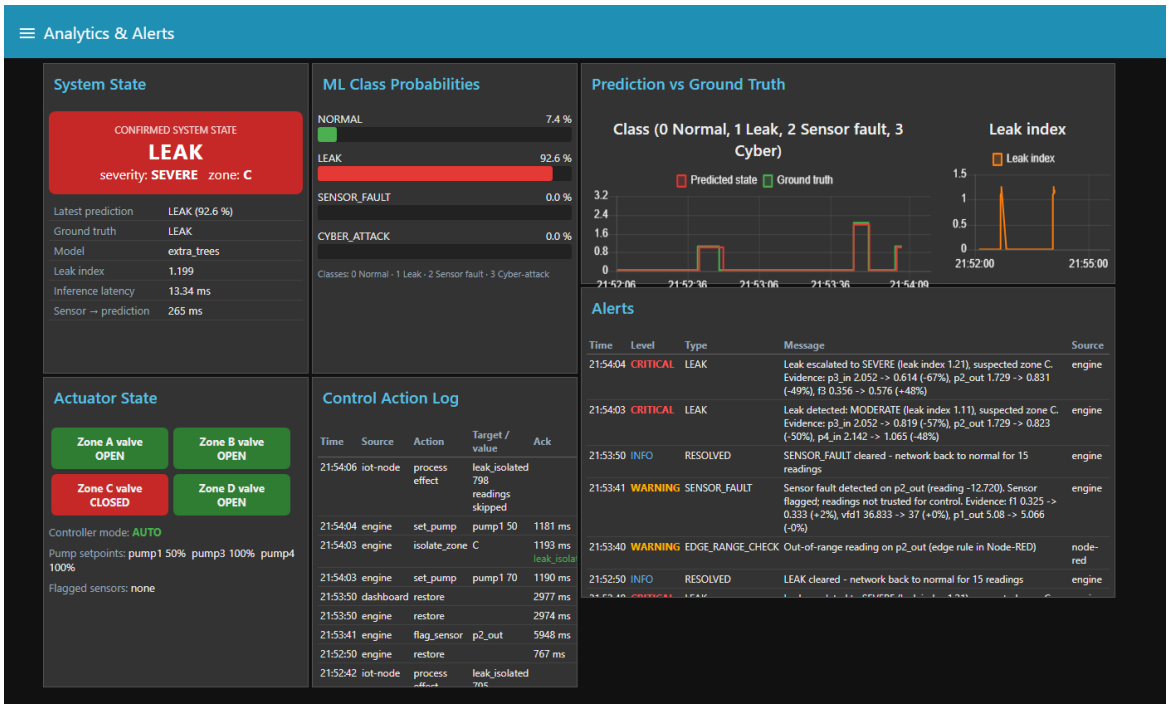



*Figure 9: Analytics page during LeakEvent1: severe leak in zone C, zone valve closed, pump at 50 %, alerts with evidence*

```
21:55:31  [DATA]    now playing 'cyber' readings 1500-2400
21:55:53  [DATA]    now playing 'leak2' readings 480-1385
21:56:00  [ALERT]   WARNING LEAK: Leak detected: MINOR (leak index 0.14), suspected zone B. Evidence: p3_in 1.693 -> 1.455 (-13%), …
21:56:00  [COMMAND] from engine: set_pump pump1 85  (minor leak - reduce pressure)
21:56:00  [CLOUD]   system state = LEAK   (model extra_trees, confidence 99%, actual LEAK)
21:56:00  [ACTUATOR] done. mode=AUTO pump1=85% closed valves=none flagged=none
21:56:01  [ALERT]   CRITICAL LEAK: Leak escalated to MODERATE (leak index 0.26), suspected zone B. Evidence: p4_in 1.842 -> 2.129 (…
21:56:02  [COMMAND] from engine: set_pump pump1 70  (moderate leak - reduce pressure)
21:56:02  [COMMAND] from engine: isolate_zone B   (moderate leak in zone B)
21:56:02  [ACTUATOR] done. mode=AUTO pump1=70% closed valves=none flagged=none
21:56:02  [ACTUATOR] done. mode=AUTO pump1=70% closed valves=['iso_B'] flagged=none
21:56:04  [EFFECT]  leak_isolated: 563 anomalous readings no longer happen
21:56:06  [ALERT]   INFO RESOLVED: LEAK cleared - network back to normal for 15 readings
21:56:07  [COMMAND] from engine: restore    (normal operation restored)
...
21:50:55  [SECURITY] REJECTED command isolate_zone B: too old (601 s, replay attack)
21:51:25  [SECURITY] REJECTED command isolate_zone A: no signature
```

*Figure 10: Device laptop console: alert, commands, actuator actions, leak contained, restore, and rejected forged commands*

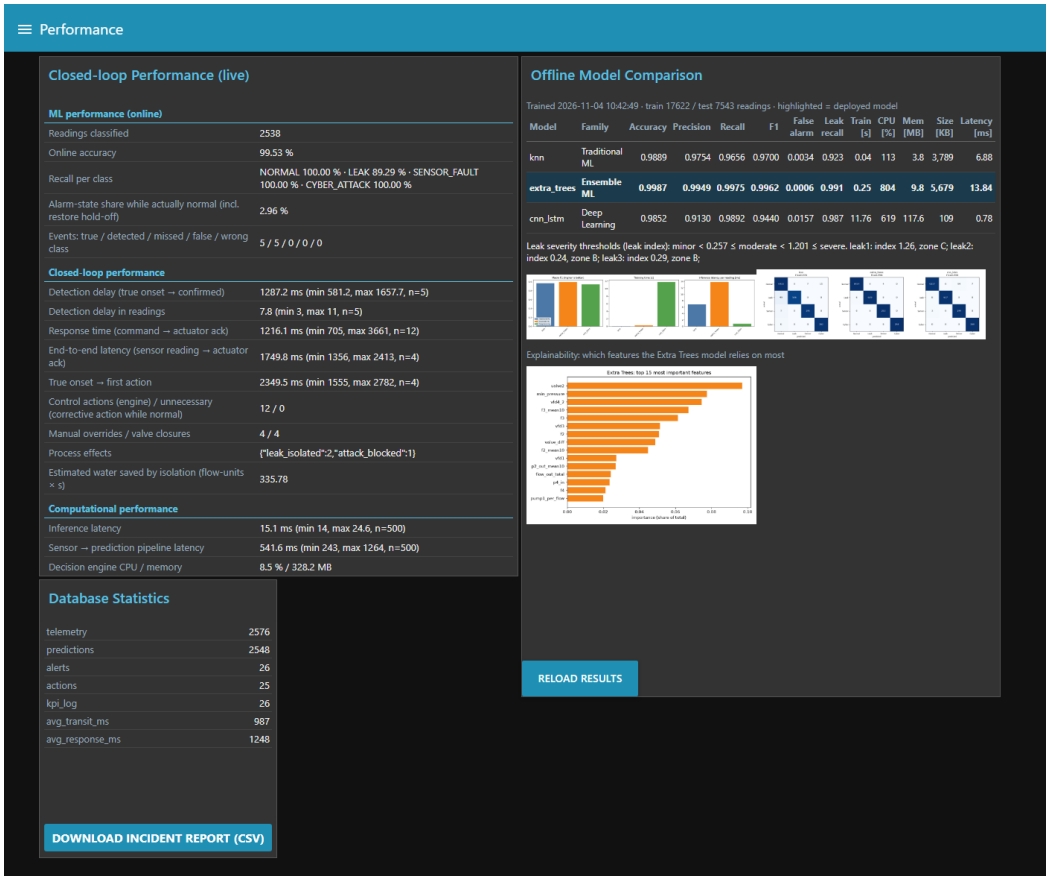



*Figure 11: Performance page: live closed-loop KPIs, model comparison and database statistics*

# 9. Limitations

- **Replayed data:** the recordings cannot respond to control actions, so their effect is approximated by skipping the rest of the anomalous run (Section 6.5). A hydraulic simulator such as EPANET/WNTR or real hardware would give true closed-loop behaviour.
- **Optimistic live accuracy:** the demo replays recordings that were largely used for training; only the held-out test set is a fair measure.
- **One testbed:** the models were trained on one network; a different network would need retraining. In these recordings valve settings also change when leaks begin; the ablation in Section 5.5 shows the models do not depend on them, but other operating points were not tested.
- **Leak location is a heuristic**, because the dataset does not record the true leak position.
- **Public broker:** HiveMQ's public broker has no access control or encryption. Commands are signed, but telemetry is readable and could be forged by anyone who knows the topics, and the shared key is stored in the project files. A production system would use a private broker with TLS and per-device credentials.
- **Latency** depends on the internet connection; the figures in Section 8 were measured on one laptop.
- **One false event** per playlist was observed with Extra Trees (a brief false sensor-fault detection).

# 10. Conclusion and Future Work

The project implements the complete prescribed architecture on two laptops connected through an online MQTT broker. Real testbed data flows from the IoT device through MQTT and Node-RED into SQLite and a live dashboard. Machine learning classifies every reading, and a decision engine acts in a closed loop. The deployed Extra Trees model reaches a macro F1 of 0.996 on held-out data, the system detected all twelve events in the end-to-end test, and forged commands are rejected by the device.

Future work:

- Couple the system to a hydraulic simulator (EPANET/WNTR) or physical sensors for true closed-loop control.
- Use a private broker with TLS and per-device credentials, and sign telemetry as well as commands.
- Add unsupervised anomaly detection to flag event types that are not in the training data.
- Train on more networks and operating points to test generalisation, and add tank-level sensing.
- Learn leak location from labelled data instead of a heuristic.

# References

[1] WDSEventDB: water-distribution system event database (dataset), provided for the course project.

[2] OASIS, "MQTT Version 3.1.1", OASIS Standard, 2014.

[3] Node-RED documentation, OpenJS Foundation, https://nodered.org/docs/.

[4] HiveMQ public MQTT broker, https://www.hivemq.com/mqtt/public-mqtt-broker/.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] P. Geurts, D. Ernst and L. Wehenkel, "Extremely randomized trees", Machine Learning, vol. 63, no. 1, pp. 3-42, 2006.

[7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory", Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[8] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, 1997.

[9] D. Mills, J. Martin, J. Burbank and W. Kasch, "Network Time Protocol Version 4", RFC 5905, 2010.

# Appendix A: How to Run

**Cloud laptop:** run `powershell -ExecutionPolicy Bypass -File start_all.ps1` in the project folder. It starts Node-RED and the decision engine and opens the dashboard at http://localhost:1880/ui.

**Device laptop:** unzip `wds_iot_device.zip` and double-click `run_device.bat` (only Python 3.8+ and internet are needed).

When the Control Panel shows "IoT node: ONLINE" the system is running. Use the scenario buttons to inject a leak, sensor fault or cyber-attack. `RUN_GUIDE.md` in the project folder gives the full steps and troubleshooting.

## Project structure

**Table 12: Main source files**

| Path | Purpose |
|---|---|
| iot_device/iot_device.py | IoT device: sensors, actuators, command verification |
| node-red/flows.json, settings.js | Node-RED flows, dashboard, database connection |
| engine/decision_engine.py | Analytics, decisions, signed commands, clock sync |
| engine/kpi.py | Closed-loop performance measurement |
| ml/train.py, deep_models.py | Model training and evaluation |
| common/features.py | Feature engineering, leak index, sensor checks |
| common/security.py | HMAC command signing |
| config.json | Broker, topics, thresholds, security settings |
| tools/build_device_zip.py, attack_demo.py | Device package builder, security demonstration |